

El motor de recomendación es una API REST construida con FastAPI y SQLite que devuelve las actividades más compatibles con el perfil de un usuario. No usa Machine Learning — es un sistema determinista basado en pesos que evolucionan con las reseñas del usuario. Cuantas más reseñas deja el usuario, más precisas son las recomendaciones.

Componente	Descripción
Lenguaje	Python 3.11
Framework	FastAPI + Uvicorn
Base de datos	SQLite — una tabla con 30 columnas de pesos por usuario
Despliegue	Docker — imagen portable, se levanta con un comando
Endpoints	2 — /recomendaciones y /actualizar-pesos

pesos

Cada usuario tiene una fila con 30 columnas — una por tag. Los pesos son números entre 0.0 y 1.0. Arrancan todos en 0.0 cuando el usuario se registra (cold-start). Solo se actualizan cuando el usuario deja una reseña.

SQL — se crea automáticamente al arrancar el servicio

```
CREATE TABLE user_weights (  
  user_id INTEGER PRIMARY KEY,  
  after_party REAL DEFAULT 0.0,  
  after_work REAL DEFAULT 0.0,  
  cuidarme REAL DEFAULT 0.0,  
  brunch REAL DEFAULT 0.0,  
  pelis REAL DEFAULT 0.0,  
  cultura REAL DEFAULT 0.0,  
  deporte REAL DEFAULT 0.0,  
  low_cost REAL DEFAULT 0.0,  
  euskaldun REAL DEFAULT 0.0,  
  arte REAL DEFAULT 0.0,  
  party REAL DEFAULT 0.0,  
  gastronomia REAL DEFAULT 0.0,  
  juegos_de_mesa REAL DEFAULT 0.0,
```

```
mercado REAL DEFAULT 0.0,
tendencia REAL DEFAULT 0.0,
conciertos REAL DEFAULT 0.0,
nocturno REAL DEFAULT 0.0,
outdoor REAL DEFAULT 0.0,
pintxopote REAL DEFAULT 0.0,
poteo REAL DEFAULT 0.0,
rooftop REAL DEFAULT 0.0,
talleres REAL DEFAULT 0.0,
teatro REAL DEFAULT 0.0,
terraceo REAL DEFAULT 0.0,
comercio_local REAL DEFAULT 0.0,
tradicional REAL DEFAULT 0.0,
chill REAL DEFAULT 0.0,
vermut REAL DEFAULT 0.0,
deportes_extremos REAL DEFAULT 0.0,
caprichos REAL DEFAULT 0.0
);
```

os pesos

Los pesos solo se actualizan cuando el usuario deja una reseña. Full Stack llama a /actualizar-pesos con las tres puntuaciones. El motor calcula la media, la normaliza a escala 0-1 y aplica la fórmula de aprendizaje con tasa $\lambda = 0.1$. Tags bien valorados suben. Tags mal valorados bajan.

Fórmula de actualización de pesos

```
# Paso 1: calcular la valoración media de la reseña
val_media = (score_servicio + score_ambiente + score_calidad_precio) / 3

# Paso 2: normalizar a escala 0.0-1.0
val_norm = (val_media - 1) / 4

# score 1 → val_norm 0.0 (muy malo)
# score 3 → val_norm 0.5 (normal)
# score 5 → val_norm 1.0 (excelente)

# Paso 3: actualizar el peso de cada tag de la actividad
peso_nuevo = peso_actual * 0.9 + val_norm * 0.1

# Ejemplo: usuario valora concierto con 5 estrellas
# peso_actual(conciertos) = 0.5
# val_norm = (5-1)/4 = 1.0
```

```
# peso_nuevo = 0.5 * 0.9 + 1.0 * 0.1 = 0.55 ← sube

# Ejemplo: usuario valora exposición con 2 estrellas

# peso_actual(arte) = 0.5

# val_norm = (2-1)/4 = 0.25

# peso_nuevo = 0.5 * 0.9 + 0.25 * 0.1 = 0.475 ← baja
```

recomendaciones

Full Stack pasa la lista de actividades disponibles en el body de la petición. El motor aplica dos filtros duros y luego calcula un score de coincidencia para cada actividad candidata.

Paso	Qué hace
Filtro 1 — time_slot	Descarta actividades cuyo time_slot no coincide con el solicitado (mañana / tarde / noche)
Filtro 2 — distancia	Descarta actividades a más de 40 km usando la fórmula de Haversine con lat/long
Score	Para cada actividad candidata: suma de pesos del usuario para los tags que coinciden / número de tags de la actividad
Ordenación	Ordena las candidatas por score de mayor a menor y devuelve las top_n (por defecto 5)

Ejemplo de scoring

```
# Ejemplo de cálculo de score

# Usuario tiene: conciertos=0.8, euskaldun=0.6, nocturno=0.4

# Actividad A tiene tags: [conciertos, euskaldun, nocturno]

score_A = (0.8 + 0.6 + 0.4) / 3 = 0.60

# Actividad B tiene tags: [conciertos, nocturno]

score_B = (0.8 + 0.4) / 2 = 0.60

# Actividad C tiene tags: [arte, cultura]

score_C = (0.0 + 0.0) / 2 = 0.00 ← no aparece en recomendaciones
```

GET /

Health check. Verifica que el servicio está activo.

Sin body

DEVUELVE

```
{ "status": "ok",
  "service": "Motor de Recomendación" }
```

POST /recomendaciones	
Devuelve las top_n actividades más compatibles con el perfil del usuario. Filtra por time_slot y distancia. Ordena por score ponderado con los pesos del usuario.	
RECIBE user_id: integer lat: float lng: float time_slot: string (mañana tarde noche) top_n: integer (opcional, default 5) activities: lista de actividades con: id, name, activity_category_id_1/2/3, time_slot, lat, lng	DEVUELVE [{ activity_id: integer name: string score: float 0-1 distancia_km: float }]
→ Full Stack pasa las activities en el body — el motor no consulta la BD de Full Stack.	

POST /actualizar-pesos	
Actualiza los pesos del usuario tras dejar una reseña. Calcula la media de las 3 puntuaciones, la normaliza a 0-1 y aplica la fórmula de aprendizaje sobre los tags de la actividad valorada.	
RECIBE user_id: integer activity_tags: string[] score_servicio: integer 1-5 score_ambiente: integer 1-5 score_calidad_precio: integer 1-5	DEVUELVE { ok: boolean weights: object (30 tags con peso) val_norm: float (valoración normalizada) tags_ignorados: string[] (tags no reconocidos) }
→ Full Stack debe llamar a este endpoint automáticamente al guardar una reseña en reviews.	
→ La valoración general no se recibe — se calcula internamente como media de las 3 puntuaciones.	

er

Comandos Docker
<pre># Construir la imagen docker build -t motor-recomendador . # Levantar el contenedor # -v monta un volumen para persistir la BD SQLite entre reinicios # -p expone el puerto 8000 docker run -v ./data:/data -p 8000:8000 motor-recomendador # La API estará disponible en: http://localhost:8000 # Documentación automática (Swagger): http://localhost:8000/docs</pre>

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

RUN mkdir -p /data

ENV DB_PATH=/data/weights.db

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt

```
fastapi==0.110.0

uvicorn==0.28.0

pydantic==2.6.4

# SQLite viene incluido en Python – no necesita instalación
```

OS

Evento	Qué ocurre
Usuario se registra	Full Stack guarda sus tags en user_preferences. El motor no hace nada todavía — los pesos arrancan en 0.0 la primera vez que interactúa.
Usuario pide recomendaciones	Full Stack llama a POST /recomendaciones con el user_id, ubicación, time_slot y la lista de activities disponibles. El motor filtra, puntúa y devuelve top 5.
Usuario deja una reseña	Full Stack guarda la reseña en reviews y llama a POST /actualizar-pesos con las 3 puntuaciones. El motor actualiza los pesos en SQLite.
Usuario vuelve a pedir recomendaciones	El motor usa los pesos actualizados. Las actividades bien valoradas antes suben en el ranking. Las mal valoradas bajan.

torio

Árbol de archivos

```
desafio_data/

■■■ main.py # API FastAPI – motor completo

■■■ Dockerfile # Imagen Docker

■■■ requirements.txt # Dependencias Python

■■■ .gitignore # Excluye .env, __pycache__, etc.

■■■ data/ # Directorio para weights.db (generado por Docker)

■■■ weights.db # BD SQLite – se crea automáticamente
```

Nota para el equipo: El motor no necesita datos reales para funcionar. Con `user_id=1` y cualquier lista de `activities` se puede probar el scoring. La documentación interactiva de todos los endpoints está disponible en `/docs` cuando el servicio está levantado.